RealEye Software Design Description

Architectural Design (Sections 6.1–6.2)

CMSI 4071 Senior Group Project

Team: Charlize, Tara, Anastasia, Anastasia, Noa

November 5, 2026

This document presents the architectural design of RealEye: the system objectives, the hardware, software, and human interfaces, the major software components and their interactions, and the architectural UML diagrams. The detailed design (Section 6.3) and database design (Section 6.4) follow in the next deliverable. Requirement numbers such as 5.3.1.2 refer to the RealEye Software Requirements Specification (SRS).

Table of Contents

6.0 Software Design Description **3**
6.1 Introduction 3
6.1.1 System Objectives 3
6.1.2 Hardware, Software, and Human Interfaces 4
6.2 Architectural Design 11
6.2.1 Major Software Components 12
6.2.2 Major Software Interactions 16
6.2.3 Architectural Design Diagrams 18

6.0 Software Design Description

6.1 Introduction

This document presents the architecture and design for the software of the RealEye project. RealEye is a Google Chrome extension, backed by a hosted analysis service, that helps internet users, particularly older adults and people with limited technical experience, assess whether text and images on a webpage may be AI-generated. When the user starts a scan, the extension extracts supported content from the current page and sends it to the RealEye server. The server runs a fine-tuned machine-learning classifier and checks any available C2PA Content Credentials and SynthID watermarks. The extension then shows each finding separately, an overall result state for every item, and inline warnings next to flagged content on the page. RealEye estimates where content came from; it does not judge whether content is true, safe, or fraudulent.

The RealEye Computer Software Configuration Item (CSCI) is divided into four Computer Software Components (CSCs), matching the SRS: the **Extension Interface CSC**, the **Page Interaction CSC**, the **Server CSC**, and the **Evaluation CSC**. Sections 6.1 and 6.2 describe the system objectives, external interfaces, and top-level structure of these components. This document records the design the team has agreed to; it serves as the reference for implementation and will be revised as the design is iterated through review and testing.

6.1.1 System Objectives

The objective of RealEye is to give everyday web users a fast, trustworthy second opinion about whether content they are reading may have been produced by AI, without requiring technical knowledge or an account. The application is built around the following goals:

- **One-click checking for non-experts.** A RealEye button in the Chrome toolbar opens a popup with a single "Start scan" action. Users choose a full-page scan or a scan of selected text or a single image. Results use plain language, every control works from the keyboard, and the interface meets WCAG 2.2 AA contrast, target-size, and zoom requirements.
- **Several independent signals instead of one verdict.** Each item receives a machine-learning **model estimate**, a **C2PA Content Credentials** finding, and a **SynthID watermark** finding, displayed separately. A versioned warning rule combines them into exactly one overall state: **Warning**, **Inconclusive**, or **Unable to analyze**. Missing credentials or watermarks never cancel a warning and are never treated as proof of human authorship.
- **Honest communication of uncertainty.** Percentages are always labeled "Model estimate." The interface states that AI detection can be wrong, and never describes content as a scam or misinformation only because it may be AI-generated. Results are presented as advisory.
- **Privacy by design.** Page content is sent for analysis only after the user starts a scan. The server processes submitted text and images in memory, never writes them to disk, and releases them within 60 seconds. No user accounts are required. Shared reports contain only the minimum fields, are reachable only through an unguessable link, and expire after seven days.
- **Measurable detection quality.** Each classifier must reach an AI-class F1 of at least 0.75 with a false-positive rate of at most 10% on held-out data at its released threshold (SRS 5.4.8 and 5.4.20 for the text classifier; SRS 5.4.9 and 5.4.21 for the image classifier). Results are recorded per release in the evaluation record (Section 6.2.1.4).

- **Maintainability through versioning.** Models, calibration values, thresholds, and the warning rule are versioned server-side and reported with every result. They can be replaced without reinstalling the extension, and all communication between the extension and server goes through one documented JSON contract.

6.1.2 Hardware, Software, and Human Interfaces

The following paragraphs identify every external interface of RealEye and describe how each one relates to the system. Paragraphs 6.1.2.1–6.1.2.7 cover hardware, 6.1.2.8–6.1.2.13 cover software interfaces and services, 6.1.2.14–6.1.2.32 cover third-party libraries, models, and datasets (one per paragraph), and 6.1.2.33 covers the human interface. The remaining open design decisions are listed in Table 2.

6.1.2.1 Client Computer

RealEye runs on the user's desktop or laptop computer in Google Chrome (6.1.2.8) on Windows 11, macOS, or ChromeOS. The minimum client is a 64-bit processor with 4 logical CPU cores, 8 GiB RAM, and a network connection (SRS 5.5.2). The client performs no machine-learning inference: it only extracts page content, sends requests, and renders results, so its load is limited to the browser itself.

6.1.2.2 Keyboard

Any standard keyboard supported by the operating system. Every RealEye workflow can be completed with the keyboard alone (SRS 5.4.11):

- **Opening RealEye:** the extension declares a keyboard shortcut for its toolbar action (the Manifest V3 `commands` entry `_execute_action`, suggested **Alt+Shift+R**, or **Option+Shift+R** on macOS). Users can change it at `chrome://extensions/shortcuts`. Keyboard users can also reach the RealEye icon through Chrome's own toolbar keyboard navigation and press **Enter**.
- **Inside the popup: Tab** and **Shift+Tab** move focus between the scope choices, Start scan, Cancel, each result item, and the Copy, Create share link, and Close results controls; **Enter** or **Space** activates the focused control; **Escape** closes the popup.
- **Choosing one image for a selected-content scan:** when the user chooses "Selected content" and no text is highlighted, the popup lists the eligible images on the page (a thumbnail, the image's alt text or file name, and its position on the page). **Up** and **Down** arrows move through the list and **Enter** selects the image to scan. The chosen image is outlined on the page so sighted keyboard users can confirm it.
- **On the page:** inline warning controls (Open explanation, Dismiss) are reachable with **Tab** in page order; **Escape** closes an open in-page explanation panel and returns focus to its warning.

Every focused control shows a visible focus indicator (SRS 5.3.1.49). Apart from the one changeable shortcut that opens the popup, RealEye defines no keyboard shortcuts, so it does not conflict with the browser's or a screen reader's shortcuts.

6.1.2.3 Mouse or Trackpad

Any pointing device supported by the operating system. The user clicks the RealEye toolbar icon to open the popup and clicks its controls. For a selected-content scan, the user highlights text by dragging on the page, or chooses a single image. On the page, the user clicks an inline warning's

Open explanation or Dismiss control. Every interactive target is at least 24 × 24 CSS pixels (SRS 5.4.13), and the page stays scrollable and clickable while results are shown (SRS 5.3.2.27).

6.1.2.4 Display

A color display of at least 1280 × 720 pixels (SRS 5.5.2). The popup is laid out for widths of 320–600 CSS pixels without clipped or overlapping labels (SRS 5.3.1.19) and stays usable at 200% browser zoom (SRS 5.4.14). Inline "Possible AI" labels are drawn on the web page outside the flagged content. All extension-owned text meets WCAG 2.2 AA contrast (4.5:1 for normal text, 3:1 for large text and control borders), and color is always paired with a text label (SRS 5.3.1.47, 5.4.12).

6.1.2.5 Analysis Server Host

The Server CSC runs on a 64-bit Linux host (Ubuntu 24.04 LTS for qualification), deployed with the Render hosting service and reachable through HTTPS. The host provides at least 4 vCPUs, 16 GiB RAM, and 30 GiB of disk (SRS 5.5.2). Inference runs on the CPU, with the released models loaded into memory at startup. A persistent volume holds the SQLite report database. A single application instance serves all requests in this baseline. During development, the same server runs locally on a team member's laptop.

6.1.2.6 Model Training Hardware

The Evaluation CSC fine-tunes and evaluates models on a Google Colab virtual machine with an NVIDIA T4 GPU (16 GB VRAM). Model artifacts and evaluation records are stored in a shared Google Drive folder. This hardware is used only offline, during development and release. It is not needed when RealEye is in use.

6.1.2.7 Network Interface

RealEye requires an active internet connection over Wi-Fi or Ethernet. All communication between the extension and the server, and all access to shared reports, uses **HTTPS** (HTTP/1.1 over TLS 1.2 or newer, TCP port 443) (SRS 5.5.2.2). The extension also uses HTTPS to retrieve original image bytes from the image's own website. During development only, the extension talks to a local server at `http://127.0.0.1:8000`. The reference network for performance testing provides at least 50 Mbps down and 20 Mbps up, with at most 100 ms round-trip time (SRS 5.5.3).

6.1.2.8 Google Chrome Browser

RealEye requires desktop Google Chrome, Stable channel, version 154.0 or later with Manifest V3 support (SDP 4.2.2; SRS 5.5.1). For qualification testing, the Stable build installed on the first qualification date is recorded and held fixed for all trials. Chrome hosts both client CSCs: it runs the popup page, injects the page scripts into the active tab, runs the extension's service worker, stores session data, and enforces the extension's permissions. Chrome does not allow extensions to run on some pages, such as `chrome://` pages and the Chrome Web Store; on those pages RealEye disables Start scan and explains why (SRS 5.3.1.7, 5.3.1.8).

6.1.2.9 Chrome Extension Platform (Manifest V3)

The Extension Interface and Page Interaction CSCs are packaged as a Chrome Manifest V3 extension. They use these Chrome APIs:

- `chrome.action` and `commands`: show the RealEye toolbar icon, open the popup, and define the keyboard shortcut that opens it.

- `chrome.tabs`: identifies the active tab and its URL for the scan.
- `chrome.scripting.executeScript`: injects the extraction and annotation scripts into the active page.
- **Extension service worker**: the background script that fetches original image bytes.
- `chrome.storage`: keeps scan state and whether the user has seen the upload disclosure.
- **Clipboard API**: copies an item's findings.
- `chrome.permissions.request`: asks for optional image-origin access when the user clicks "Allow image access."

The required permissions are limited to `activeTab`, `scripting`, `storage`, `clipboardWrite`, and host access to the RealEye HTTPS service. HTTP and HTTPS image origins are declared only as optional host permissions (SRS 5.3.1.41).

6.1.2.10 RealEye REST API

The extension and server communicate through a REST API implemented with FastAPI. Requests and responses are JSON (`application/json`, UTF-8) over HTTPS. The endpoints are:

- `POST /analyze`: analyzes one batch of at most 10 items, with a request body of at most 16 MiB.
- `GET /health`: reports service status.
- `POST /reports`: creates a shared report snapshot for one item.
- `GET /r/{id}`: displays a shared report.

Images are sent as base64-encoded original bytes inside the JSON, so that C2PA credentials remain intact. Errors use one format, `{"error": {"code", "message"}}`, with HTTP status codes 400 (no supported items or malformed request), 401 (administrative or configuration request without a valid server-side administrator credential, SRS 5.3.3.75), 403 (browser origin not allowed), 413 (body over 16 MiB), 422 (rule violation such as more than 10 items), and 429 (rate limit, with a `Retry-After` header). The agreed contract is maintained in `docs/api-contract.md`, and detailed message formats are specified in Section 6.3.

6.1.2.11 C2PA Content Credentials Verification

Interface. For JPEG, PNG, and static WebP images submitted with their original, unmodified bytes (at most 10 MiB), the C2PA and Accessible SynthID CSU inspects any embedded C2PA (Coalition for Content Provenance and Authenticity) manifest. It does this through an in-process library call: the input is the image bytes and MIME type; the output is the manifest's validation result and its assertions. The CSU maps this to exactly one status: verified AI assertion, verified credential without an AI assertion, missing credential, invalid credential, not applicable (text items), unavailable (original bytes not supplied), or error. A missing credential is a completed inspection; an invalid credential is not (SRS 5.1.1). The check has a five-second deadline. The trust list used to validate signing certificates is part of the release configuration.

Selected library: c2pa-python 0.38.2 (Content Authenticity Initiative; MIT or Apache-2.0 license; requires Python 3.10 or newer, so it runs on the server's Python 3.11). It wraps the reference C2PA implementation and reads and validates C2PA manifests from JPEG, PNG, and WebP files, including signature checks against a configurable trust list without network access. It was chosen because it is the C2PA project's own maintained Python binding. RealEye uses **c2pa-python version 0.38.2**. This exact version will be installed when C2PA verification is integrated, and any later upgrade will be re-tested with the valid, missing, and invalid credential test files (SRS 5.5.1.4).

6.1.2.12 SynthID Watermark Detector

When Google DeepMind provides an authorized programmatic SynthID detector for an item type, the server calls it over HTTPS. The detector's credentials are kept only in server-side configuration (SRS 5.3.3.76). Each check has a five-second deadline. If no authorized detector is available, or the check times out, the result is "unavailable." Item types that SynthID does not support return "not applicable."

6.1.2.13 Shared Report Storage (SQLite 3)

Shared reports are stored in a SQLite 3 database file on the server's persistent volume, accessed through Python's built-in `sqlite3` module. It is the only persistent data store in RealEye. It holds report snapshots only, never the scanned text or images. An hourly cleanup task deletes snapshots older than seven days. The database design is specified in Section 6.4.

6.1.2.14 React 18.3.1

Purpose: Builds the popup user interface as components: scan controls, progress, the results list, explanations, and copy and share controls. **Used by:** Extension Interface CSC (`Popup.tsx`).

6.1.2.15 React DOM 18.3.1

Purpose: Renders the React components into the popup page (`index.html`). **Used by:** Extension Interface CSC (`main.tsx`).

6.1.2.16 TypeScript 5.6.3

Purpose: The typed language for all extension source code. The API request and response types mirror the server's schemas, so mismatches are caught at build time. **Used by:** Extension Interface and Page Interaction CSCs.

6.1.2.17 @types/chrome 0.0.287

Purpose: Type definitions for the Chrome extension APIs (`chrome.tabs`, `chrome.scripting`, `chrome.storage`, `chrome.permissions`). **Used by:** Extension Interface and Page Interaction CSCs, at build time only.

6.1.2.18 Vite 5.4.11

Purpose: Compiles and bundles the extension into the `extension/dist` folder that Chrome loads. **Used by:** The build process for both client CSCs. Not part of the installed extension.

6.1.2.19 @vitejs/plugin-react 4.3.4

Purpose: Lets Vite compile React's JSX syntax. **Used by:** The extension build process.

6.1.2.20 Node.js 22

Purpose: Runs Vite, TypeScript, and the npm package manager. A committed `package-lock.json` pins every dependency version. **Used by:** Development and build environment only.

6.1.2.21 FastAPI 0.115.6

Purpose: Defines the HTTP endpoints (`POST /analyze`, `GET /health`, `POST /reports`, `GET /r/{id}`), routes requests, and returns errors in the contract's format. **Used by:** Analysis API CSU and the report endpoints of the Sharing and Report Viewer CSU (`server/app/main.py`).

6.1.2.22 Pydantic 2.14.1

Purpose: Declares and validates the request and response schemas: types, allowed status values, required fields, and the 10-item limit. **Used by:** Analysis API CSU (`server/app/schemas.py`).

6.1.2.23 Starlette 0.41.3

Purpose: The web toolkit FastAPI is built on; provides the request, response, and middleware layer used for the origin check, rate limiting, and request timing. **Used by:** Analysis API and Operations and Monitoring CSUs. Installed with FastAPI and pinned in the lockfile.

6.1.2.24 Uvicorn 0.32.1

Purpose: The ASGI web server that runs the FastAPI application and accepts HTTP connections. In deployment, HTTPS is terminated by the hosting platform in front of Uvicorn. **Used by:** Server CSC at runtime.

6.1.2.25 PyTorch 2.6.0

Purpose: Executes the neural-network classifiers. **Used by:** Training CSU (GPU fine-tuning) and Text and Image Inference CSU (CPU inference on the server).

6.1.2.26 Hugging Face Transformers 4.49.0

Purpose: Loads pretrained models and tokenizers and provides the `Trainer` fine-tuning loop. **Used by:** Training CSU (`train_text.py`), Evaluation CSU (`analyze_errors.py`, `calibrate.py`, `evaluate_test.py`), and Text and Image Inference CSU.

6.1.2.27 Hugging Face Datasets 3.1.0

Purpose: Downloads datasets from the Hugging Face Hub and reads the prepared JSON Lines files for training. **Used by:** Dataset and Training CSUs (`mage.py`, `train_text.py`).

6.1.2.28 pandas 2.2.3

Purpose: Cleans, deduplicates, balances, and splits the labeled data, and builds evaluation tables. **Used by:** Dataset and Evaluation CSUs (`prepare_data.py`, `analyze_errors.py`, `calibrate.py`, `evaluate_test.py`).

6.1.2.29 Accelerate 1.4.0

Purpose: Required by the Transformers `Trainer` to place training on the available GPU. **Used by:** Training CSU.

6.1.2.30 matplotlib 3.10.0

Purpose: Draws the reliability (calibration) charts saved with each evaluation. Version 3.10.0 is the one in the Google Colab environment used for the current evaluation; it is recorded with each evaluation run. **Used by:** Evaluation CSU (`calibrate.py`, `evaluate_test.py`).

6.1.2.31 Pretrained Models and Datasets

The text classifier is fine-tuned from `distilbert/distilroberta-base` on the MAGE dataset (`yaful/MAGE`), which contains human-written and AI-generated text across ten writing types, plus a test split with additional unseen writing types and generators. Both come from the Hugging Face Hub. The revisions used for the current release are:

- `distilbert/distilroberta-base`: commit `fb53ab8802853c8e4fbdbcd0529f21fc6f459b2b` (last changed February 19, 2024).
- `yaful/MAGE`: commit `342663f0a2b775455c023f5d36a1341ff0ec5402` (last changed May 22, 2024).

Neither repository has changed since before training began (October 2026), so these are the revisions the release was trained and tested on. Each release records them in its evaluation record and release manifest, and the training and data-preparation code loads them by these exact revisions. The image classifier's pretrained model and labeled dataset are an open design decision (Table 2); they will be recorded the same way, with their sources, revisions, and permitted uses, before training (SRS 5.3.4.3, 5.3.4.7).

6.1.2.32 Language Identification Library

Interface. Before text inference, the Analysis API CSU passes the normalized text to a language-identification function, in-process. The output is a language code and a confidence. If the text is not identified as English, the item receives classifier status "unsupported" with the reason "language", and no model estimate (SRS 5.3.3.23, 5.3.3.71). The decision threshold is part of the release configuration.

Selected library: lingua-language-detector 2.1.1 (lingua-py; Apache-2.0 license; requires Python 3.10 or newer). Version 2.1.1 is used because the newer 2.2.0 requires Python 3.12, while the server runs Python 3.11. The detector is built once at server startup in high-accuracy mode for a fixed set of common languages; a text is treated as English when English is the top result and its confidence meets the threshold in the release configuration. It was chosen over fastText `lid.176` because it needs no separately downloaded model file, runs fully offline on the CPU, and is accurate on 200–5000-character text. RealEye uses **lingua-language-detector version 2.1.1**. This exact version will be installed when language checking is integrated.

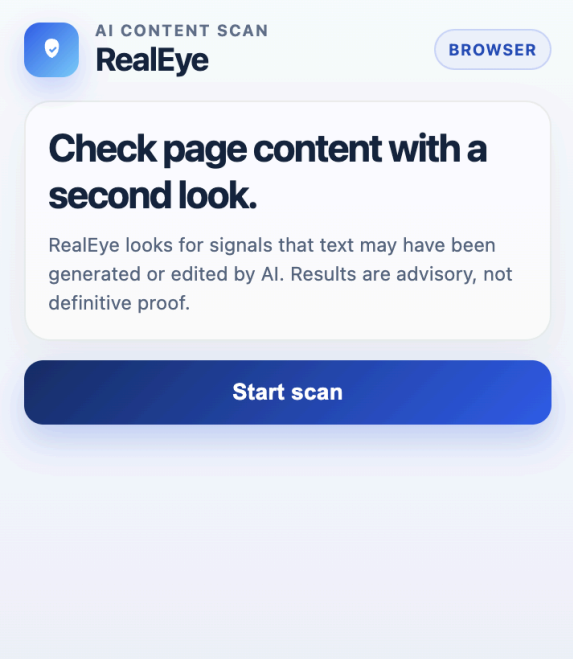
6.1.2.33 Human Interface

Users interact with RealEye through three visual surfaces:

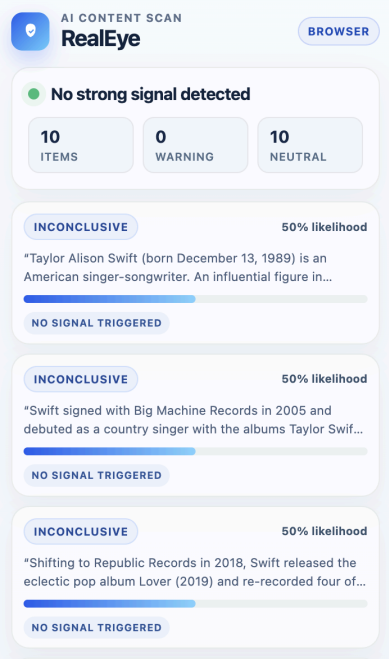
- **Extension popup**, opened from the toolbar icon (Figure 1 shows the current prototype and a drawing of this design). It moves through these states:
  - **Idle:** a short description, the scan-scope choices ("Full page" or "Selected content") with what each includes, the upload disclosure, and the **Start scan** button.
  - **Scanning:** a spinner with "Scanning…", the number of items found, and **Cancel**.
  - **Results:** a **Close results** button that dismisses the results panel (SRS 5.3.1.18), a summary card with counts of analyzed, flagged, and unavailable items, then a list of every analyzed item. Each item shows its type, its overall state with a text label, "Model estimate: N%", separate C2PA and SynthID findings, the scan time and source page, an explanation, the detection-limits notice, and **Copy** and **Create share link** actions.
  - **Error:** for example "Service unavailable", with **Try again**.
  - **Help view:** explains result states, uncertainty, missing signals, and each browser permission.
- **Inline page indicators:** a flagged item on the page gets a border and a "Possible AI" label placed outside its content, with **Open explanation** and **Dismiss** controls. The page stays scrollable and clickable while results are shown.

- **Shared report page:** a read-only HTML page at `/r/{id}`, viewable without an account. It shows the item type, source page, scan time, overall state, separate findings, and detector limitations, or an "expired" message after seven days.

All extension-owned UI provides visible keyboard focus, accessible names for icon-only controls, an accessible live region announcing scan completion and failure, color always paired with text labels, contrast of at least 4.5:1, interactive targets of at least 24 × 24 CSS pixels, and usable layout at 200% zoom (SRS 5.3.1.47–5.3.1.51, 5.4.11–5.4.14).



(a) Prototype: before a scan



(b) Prototype: results

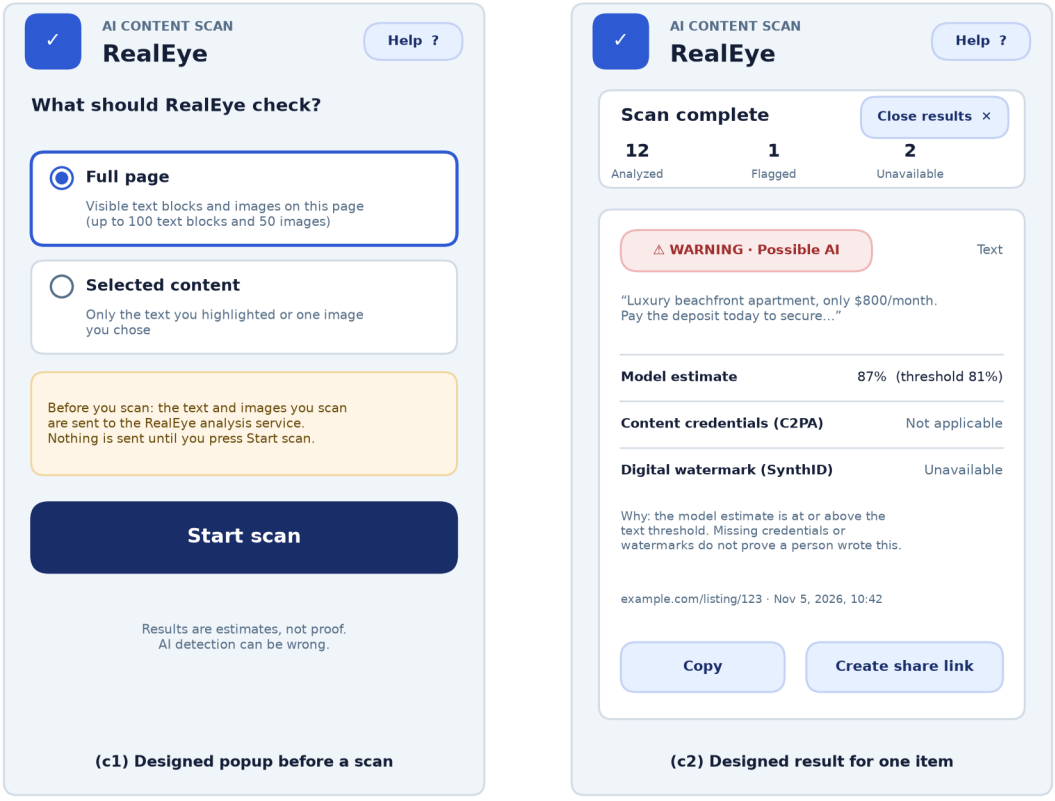


Figure 1. RealEye extension popup. (a)–(b) Current prototype, November 2026. (c1)–(c2) Simulated drawing of the designed interface described in this section, with scope choices, the upload disclosure, a separately labeled model estimate, and separate C2PA and SynthID findings.

6.2 Architectural Design

RealEye uses a **client–server architecture with an offline model pipeline**. The design has three runtime parts and one offline part:

- **A thin browser client:** the Extension Interface and Page Interaction CSCs. It handles the user interface and access to the web page.
- **A stateless analysis service:** the Server CSC. It holds all machine-learning models, evidence checks, and decision rules.
- **A small persistent store:** used only for shared reports.
- **An offline Evaluation CSC:** prepares data, trains, calibrates, and qualifies models, then publishes them to the server as versioned release artifacts.

This partitioning follows from the requirements:

- Models are too large to run reliably inside a browser extension.
- Provider credentials must stay on the server (SRS 5.3.3.76).
- Models and thresholds must change without reinstalling the extension.

Coupling is kept low:

- The client and server share exactly one versioned JSON contract.

- Inside the server, the Analysis API CSU acts as a coordinator, and each other CSU exposes a single, narrow function interface to it.
- The evidence checks (classifier, C2PA, SynthID) are independent, so one can fail or time out without blocking the others (SRS 5.3.3.10).
- The warning rule is separate from the models, so thresholds can be retuned without retraining.

Cohesion is high: each CSU owns one responsibility from the SRS breakdown. One unit is added to the SRS breakdown: an **Operations and Monitoring CSU** in the Server CSC, which provides health checks, privacy-safe logging, and operational metrics in response to the SRS review.

CSC	Runs on	Responsibility	Repository location
Extension Interface	Chrome popup (user's computer)	Scan controls, progress, results, explanations, accessibility, copy and share	`extension/src/` (`Popup.tsx`, `api.ts`, `main.tsx`), `extension/public/manifest.json`
Page Interaction	Content scripts and service worker in Chrome	Find and extract page text and images; draw and remove inline warnings	`extension/src/` (`extract.ts`, annotation script, service worker)
Server	Linux host (FastAPI, Uvicorn)	Validate requests, run classifiers and evidence checks, apply the warning rule, serve shared reports, monitoring	`server/app/` (`main.py`, `schemas.py`, and one module per CSU)
Evaluation	Google Colab GPU (offline)	Prepare labeled data, fine-tune, calibrate, choose thresholds, test, and release models	`ml/text/` (and `ml/image/` for the image classifier)

Table 1. RealEye CSCs, where they run, and where they live in the repository.

The following design decisions are still open. Each has an interface that is already fixed by this architecture, so the choice can be made without changing other components.

Decision	Options	Selection criteria	Decide by
Image classifier base model and dataset (6.1.2.31)	Pretrained image classifiers on the Hugging Face Hub; labeled human and AI image datasets	Permitted use; class balance; meets F1 and false-positive targets on held-out data; CPU inference time	Before image training (SDP: Nov 24, 2026)
SynthID detector access (6.1.2.12)	Authorized programmatic detector, if available; otherwise "unavailable"	Access permitted for programmatic use; supported media types	Before integration (SDP: Nov 26, 2026)

Table 2. Open design decisions.

6.2.1 Major Software Components

Each CSC below lists its Computer Software Units (CSUs) and the SRS functional requirements it covers (SRS 5.3.1–5.3.4), so that every functional requirement is assigned to a component. Table 3 summarizes the coverage.

6.2.1.1 Extension Interface CSC

The Extension Interface CSC is everything the user sees in the popup. It owns the **scan lifecycle**: idle → scanning → completed, failed, or canceled. It also creates the scan and item identifiers, sends content to the server in batches, and turns server responses into labeled, accessible results. It does not make detection decisions; it displays what the server returns.

- **Popup and Controls CSU:** the toolbar entry point, the choice between "Full page" and "Selected content" with a description of each, the upload disclosure, the **Start scan** action (enabled only on pages Chrome allows), progress and cancellation, the scan-limit notice, the help view, and the "Allow image access" permission prompt.
- **Results and Accessibility CSU:**
  - the completion summary and the list of every analyzed item;
  - for each item, separately labeled model-estimate, C2PA, and SynthID findings, the overall state, the evidence behind any warning, and the detection-limits notice;
  - "Stale" labeling when the page changes;
  - Copy (with "Copied" or "Copy failed" feedback), share preview and link creation, and "Service unavailable" or "Failed" states with retry;
  - keyboard focus, accessible names, and live-region announcements.

Requirements covered: SRS 5.3.1.1–5.3.1.58; performance targets 5.4.1, 5.4.4, 5.4.7, 5.4.11–5.4.14, and 5.4.19.

6.2.1.2 Page Interaction CSC

The Page Interaction CSC is the code that runs inside, or on behalf of, the web page being scanned. It turns a page into a list of eligible scan items, and later places and removes warning indicators on the page.

- **Selection and Extraction CSU:**
  - finds text blocks in visible `p`, `li`, `h1`–`h6`, `blockquote`, `pre`, and `figcaption` elements, or uses the highlighted selection;
  - normalizes whitespace, removes empty and duplicate text, and marks text outside 200–5000 characters as skipped;
  - finds images and removes duplicate image URLs, applying the caps of 100 text blocks and 50 images in document order;
  - reports inaccessible frames;
  - has the extension service worker retrieve original image bytes, only for origins the user has granted, without recompressing them.
- **Page Annotation CSU:** locates each scanned element on the page, adds the "Possible AI" border and label outside the flagged content, wires up the **Open explanation** and **Dismiss** controls, and removes outdated indicators when a new scan replaces an old one or an element disappears.

Requirements covered: SRS 5.3.2.1–5.3.2.39; performance target 5.4.2.

6.2.1.3 Server CSC

The Server CSC is the hosted analysis service. It receives batches of items, produces independent findings for each, combines them with the versioned warning rule, and returns one result per item. It also hosts shared reports. It keeps no user accounts and stores no scanned content.

- **Analysis API CSU:**
  - validates each request: body size (16 MiB), item count (10), item type, text length and language, image format and size limits, the browser `Origin` allowlist, and the rate limit of 60 requests per minute per IP;
  - dispatches each item to the applicable checks;
  - formats responses that keep item identifiers, rule and model versions, and explicit "not applicable", "unavailable", and "error" statuses;
  - keeps content and URLs out of logs, and releases content within 60 seconds.
- **Text and Image Inference CSU:** runs the released classifiers.
  - **Text:** every text item first passes the 200–5000-character length check and the English check on the full text. It is then tokenized exactly as in training, and the model scores the **first 512 tokens**. This is the same input the model was trained, calibrated, and tested on, so the released threshold and test results apply. The released Platt calibration (a, b) turns the model's raw output into an estimate between 0 and 1, plus a class label.
  - **No silent truncation:** the analyzed portion is measured, not estimated. The tokenizer reports where its last scored token ends in the original text (excluding the special start and end tokens), and that character count is returned with the result. The estimate describes **only the analyzed portion**. When the text is longer, the explanation says so (for example, "This estimate is based on the first 2,040 of 4,800 characters"), so the result never implies the whole text was checked. Scoring long text in overlapping pieces is deferred, because it would require new calibration and a new held-out test.
  - **Images:** the image classifier uses its own preprocessing, calibration, and threshold, taken from its own validated release configuration. It does not reuse the text model's settings.
  - The model version module reports which artifact and release configuration produced each estimate.
- **C2PA and Accessible SynthID CSU:** verifies C2PA credentials on original image bytes, checks whether an authorized SynthID detector exists for the item type and runs it with a five-second deadline, and records each method's status and limitation.
- **Warning Interpretation CSU:** compares each estimate with its validated, modality-specific threshold and applies the versioned rule from SRS 5.1.1:
  - **Warning** if any trigger is present: a classifier estimate at or above its threshold, a verified C2PA AI assertion, or a positive SynthID result.
  - Otherwise **Inconclusive** if at least one check **completed**. These count as completed: a classifier result with an estimate; a C2PA inspection with a verified result or a missing credential; a SynthID check with a positive or negative result.
  - These do **not** count on their own: an invalid credential, unavailable, error, not applicable, or unsupported.
  - Otherwise **Unable to analyze**.
  - It sets the conflict flag when a trigger is present together with a completed below-threshold estimate, and attaches an explanation of the evidence used.
- **Sharing and Report Viewer CSU:** when the user chooses Create share link, it stores a minimal snapshot under a random identifier with at least 128 bits of entropy. It serves the read-only report page and enforces seven-day expiry, with hourly deletion of expired snapshots. This is the refined form of the **Trusted Circle** idea from the project proposal:

instead of accounts, contact lists, and invitations, the user sends the link to a family member or trusted person themselves, and RealEye never sends messages on the user's behalf (SRS 5.3.3.64). Account-based sharing remains a deferred extension (SRS 5.5.4).

- **Operations and Monitoring CSU (added):**
  - `GET /health` reports service status and the loaded model and rule versions;
  - structured operational logs record timings, status codes, timeouts, and rate-limit rejections, never submitted text, image bytes, source URLs, or share tokens;
  - counters let maintainers check the latency and concurrency targets;
  - administrative functions require a server-side administrator credential.

Requirements covered: SRS 5.3.3.1–5.3.3.82; performance targets 5.4.3, 5.4.5, 5.4.6, and 5.4.15–5.4.18; runtime use of the thresholds behind 5.4.8, 5.4.9, 5.4.20, and 5.4.21. The Operations and Monitoring CSU supports 5.3.3.28, 5.3.3.66, 5.3.3.75, and the SRS review's request for monitoring and diagnostics.

6.2.1.4 Evaluation CSC

The Evaluation CSC is the offline pipeline that turns labeled data into a qualified, versioned model release. It never runs on the server at runtime.

- **Dataset CSU:** records each dataset's source and permitted use. Its partition and duplicate-isolation module (`prepare_data.py`, `mage.py`):
  - keeps text of 200–5000 characters and removes exact duplicates across the training, validation, and test splits;
  - balances human and AI examples within each writing type, so the model does not learn writing type as a shortcut;
  - writes a manifest of counts and the random seed.
- **Training CSU:** records the base model and configuration, then fine-tunes the classifier on the training split (`train_text.py`). It uses label smoothing to limit overconfidence and keeps the epoch with the lowest validation loss. It exports an inference-ready model, its tokenizer, and a metrics record that includes the per-epoch loss history.
- **Evaluation CSU:**
  - **Threshold validation** (`calibrate.py`): fits Platt calibration on one half of the validation set and chooses the warning threshold on the other half for a target false-positive rate, using validation data only.
  - **Held-out testing** (`evaluate_test.py`): runs once on the untouched test split with the frozen calibration and threshold, after a leak check.
  - **Metrics reporting** (`analyze_errors.py` and the test record): reports the **confusion matrix** (true and false positives and negatives), precision, recall, F1, false-positive rate, class counts, calibration error, and results by writing type, for each classifier on held-out test data (SRS 5.3.4.9, 5.3.4.10).

The output of this CSC is a **versioned release**: the model weights and tokenizer, `calibration.json` (Platt parameters a and b and the cutoff), and the evaluation record. The Server CSC deploys this release.

Current text-model evaluation record. Release candidate `distilroberta-base-20261030-0247` (DistilRoBERTa fine-tuned on MAGE, Platt calibration, cutoff 0.8118 chosen for a 5% target

false-positive rate on validation data) was tested once on the 51,002-text MAGE test split on November 4, 2026: **F1 0.893** and a **5.9% false-positive rate**, meeting SRS 5.4.8 and 5.4.20. The full record, including results by writing type, calibration error, and the leak check, is in `ml/text/ERROR_ANALYSIS.md` and the release's `test_results.json`.

Requirements covered: SRS 5.3.4.1–5.3.4.14; performance targets 5.4.8–5.4.10, 5.4.20, and 5.4.21.

CSC	CSUs	Functional requirements	Performance requirements
Extension Interface	Popup and Controls; Results and Accessibility	5.3.1.1–5.3.1.58	5.4.1, 5.4.4, 5.4.7, 5.4.11–5.4.14, 5.4.19
Page Interaction	Selection and Extraction; Page Annotation	5.3.2.1–5.3.2.39	5.4.2
Server	Analysis API; Text and Image Inference; C2PA and Accessible SynthID; Warning Interpretation; Sharing and Report Viewer; Operations and Monitoring	5.3.3.1–5.3.3.82	5.4.3, 5.4.5, 5.4.6, 5.4.15–5.4.18
Evaluation	Dataset; Training; Evaluation	5.3.4.1–5.3.4.14	5.4.8–5.4.10, 5.4.20, 5.4.21

Table 3. Functional and performance requirement coverage by CSC.

6.2.2 Major Software Interactions

This section describes how the components communicate. The exact message fields and formats are specified in the detailed design (Section 6.3).

6.2.2.1 Scan lifecycle and ownership

The **popup owns each scan** from start to finish. When the user activates Start scan, the popup creates the scan identifier, extracts the items through the page scripts, keeps the queue of pending batches, and keeps a handle on the request in flight so it can be aborted.

- **Cancel:** aborts the in-flight `/analyze` request, sends no further batches, ignores any late response that belongs to a canceled scan, and removes pending results from the display within one second (SRS 5.4.7). Results already completed stay visible and are labeled as partial.
- **Closing the popup:** Chrome ends the popup's page, which ends the scan in the same way as Cancel. Completed results and their page URL are saved in the extension's session storage, so reopening the popup on the same page shows them again; if the page URL or state no longer matches, they are labeled "Stale" (SRS 5.3.1.15).
- **Inline warnings:** when results arrive, the popup gives the annotation script each flagged item's explanation summary (overall state, separate findings, scan time, and the detection-limits notice). The annotation script then owns the inline indicators: it **opens and closes the on-page explanation panel itself** and **removes an indicator locally** when the user chooses Dismiss. These controls therefore keep working after the popup closes. If the popup is still open, the annotation script also sends the event to it, so the popup can highlight the same item or mark it dismissed. Copy and Create share link remain in the popup.

- **Server side:** the server does not track scans across requests. Each `/analyze` request is independent, so cancellation only needs to stop the client from sending more batches. Content from an aborted request is released within 60 seconds like any other (SRS 5.3.3.80).

6.2.2.2 Popup and page scripts

When the user starts a scan, the popup identifies the active tab and uses `chrome.scripting.executeScript` to run the extraction script inside that page. The script returns a serialized list of eligible items (normalized text, image URLs, and an element locator for each) or a reason the page cannot be scanned. After results arrive, the popup injects the annotation script with the results that need inline warnings. The annotation script handles **Open explanation** and **Dismiss** on the page by itself (see 6.2.2.1). When the popup is open, it also sends each event to the popup through Chrome extension messaging so the two stay in sync; when the popup is closed, nothing is lost, because the page script already has the explanation it needs.

6.2.2.3 Popup and service worker

For image items, the popup sends the image URLs from the current scan to the extension service worker through Chrome extension messaging. The service worker:

- accepts only URLs that belong to the user-initiated scan in the active tab;
- checks that the user granted access to each image's origin;
- fetches the original bytes over HTTPS, stopping if a redirect leads to an origin without permission;
- returns the bytes and their file type (MIME type), or a "permission denied" or "unreadable" status.

6.2.2.4 Extension and analysis server

The extension calls `POST /analyze` over HTTPS with JSON request and response bodies:

- A full-page scan is split into batches that share one scan identifier. Each batch has **at most 10 items and a request body of at most 16 MiB**, including the encoding overhead of images. The popup adds items to a batch until either limit would be exceeded; large images therefore travel in smaller batches. An image too large to fit in any request is marked unsupported in the popup with the size limit, instead of being sent.
- Each result is matched back to its page item by its item identifier, and the extension shows partial results as batches complete.
- Network failures and server error codes are turned into "Service unavailable" or "Failed" states with a retry option, within 35 seconds (SRS 5.4.19).
- No page content is sent before the user activates Start scan.
- When the user creates a share link, the extension calls `POST /reports` with only the approved fields for that one item.

6.2.2.5 Interactions inside the server

Inside the server, the Analysis API CSU coordinates the other CSUs through in-process Python calls. For each item:

- Request validation runs first.

- The classifier and the C2PA and SynthID checks then run independently and concurrently. External checks have a five-second deadline, and a check that misses it reports "unavailable" without suppressing a completed classifier result.
- The Warning Interpretation CSU combines the findings into the overall state.
- The response formatter builds the item result.

The Operations and Monitoring CSU wraps every request to record timing and status. It receives only metadata, never content.

6.2.2.6 Server and SQLite

The Sharing and Report Viewer CSU is the only component that accesses the database. It inserts a snapshot when a share link is created, selects it when `GET /r/{id}` is requested, and deletes expired snapshots in a scheduled hourly task. All statements use parameterized SQL through Python's `sqlite3` module.

6.2.2.7 Server and external evidence services

C2PA verification is an in-process library call on the original image bytes. SynthID detection, when authorized, is an outbound HTTPS call from the server, with credentials that never leave the server. Both report one of the defined statuses, and neither is ever called by the extension directly.

6.2.2.8 Evaluation CSC and Server CSC (model release)

The Evaluation CSC and the server have no runtime network connection. A release works like this:

- The Evaluation CSC exports a versioned release folder containing the model weights and tokenizer, `calibration.json` (a, b, cutoff), and the evaluation record.
- A release step copies that folder to the server's artifact location.
- The Text and Image Inference CSU loads it into memory when the server starts.
- The model and rule versions are returned with every analysis response and by `GET /health`, so each result can be traced to the evaluation that qualified it.

6.2.2.9 Evaluation CSC and external data sources

During development, the Evaluation CSC downloads pretrained models and datasets from the Hugging Face Hub and stores trained artifacts and evaluation records in the team's Google Drive folder.

6.2.2.10 Share-link recipients

A person who receives a share link opens `GET /r/{id}` in any web browser over HTTPS. The Sharing and Report Viewer CSU returns a read-only HTML page, or an "expired or unavailable" page. No account or extension is required.

6.2.3 Architectural Design Diagrams

Figures 2–5 show the architecture from four UML viewpoints: what users can do (use case), where the software runs and how the parts connect (component and deployment), the main modules and their dependencies (class and package), and how a scan moves through the system (activity with swim lanes).

Use case diagram. Figure 2 shows the three actors. The **User** chooses a scan scope, starts and cancels scans, reviews explanations, dismisses inline warnings, copies results, creates share links,

views help, and grants image access. Starting a scan always includes extracting page content and analyzing items. Analysis runs only the checks that apply to each item; unsupported or unavailable methods return their defined status (see the note). When the scan includes images, **Retrieve original images** extends content extraction, and **Allow image access** extends image retrieval only when an image's origin has not yet been granted. A **Share-link recipient** can only open a shared report. The **Developer/administrator** trains and qualifies models, releases a model with its threshold, and checks service health.

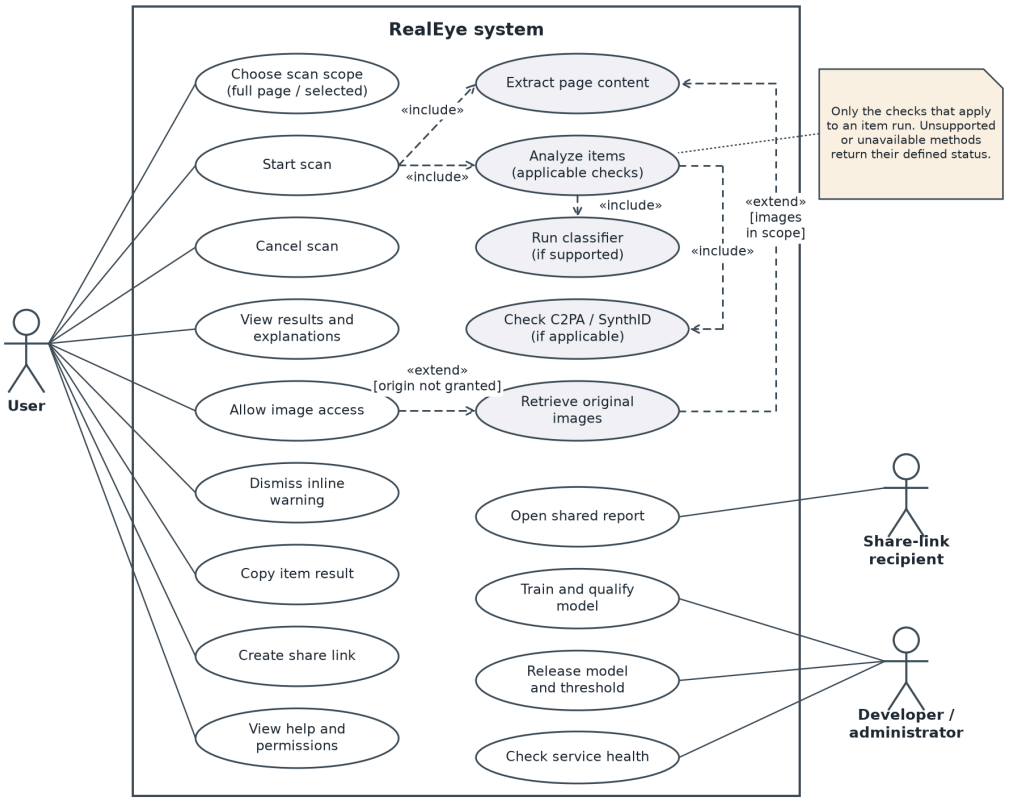


Figure 2. Use case diagram.

Component and deployment diagram. Figure 3 shows the three devices:

- **The user's computer:** runs Chrome with the popup, content scripts, and service worker.
- **The Linux server:** runs the FastAPI application with its six server CSUs, the deployed model artifact, and the SQLite report store.
- **The Colab training environment:** runs the Evaluation CSC offline and deploys a versioned model release to the server.

External systems (websites, the Hugging Face Hub, the SynthID detector, and recipients' browsers) are shown with dashed borders, and every connection between devices uses HTTPS.

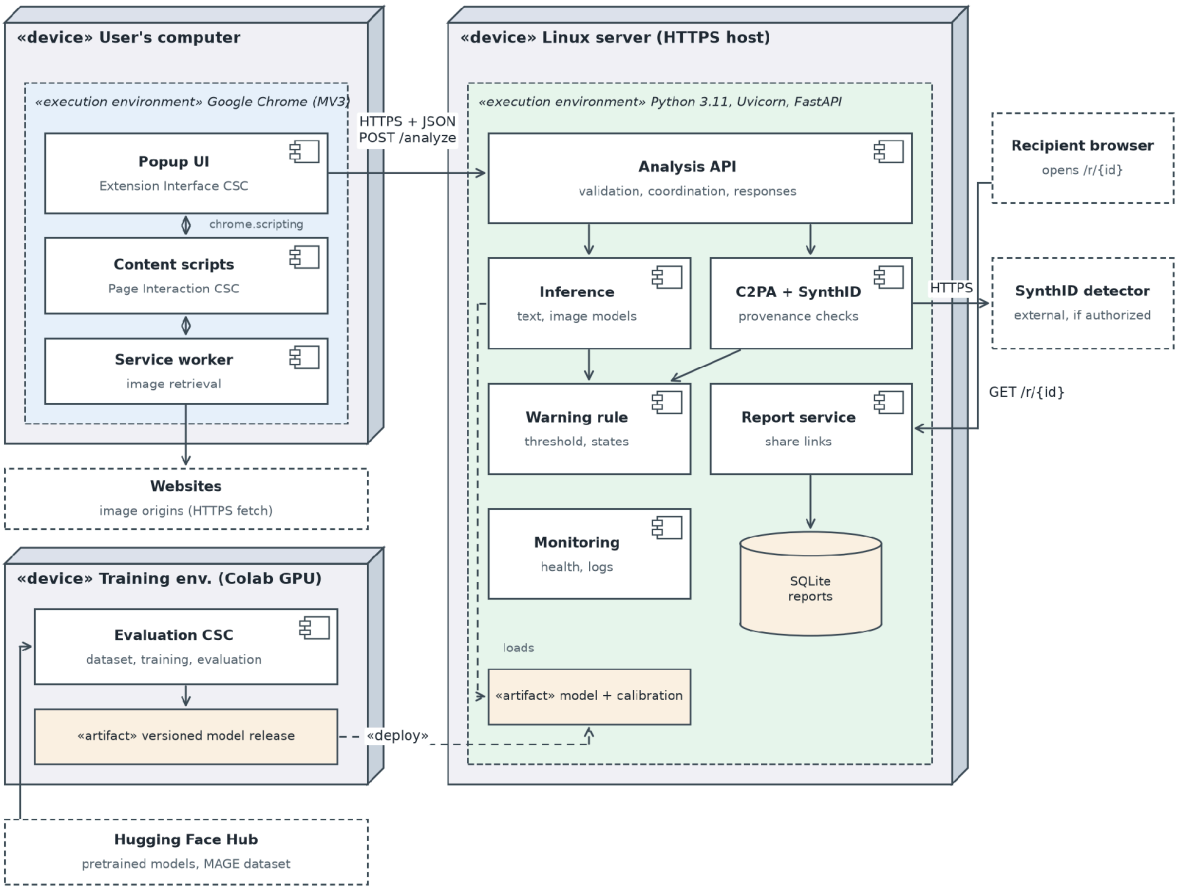


Figure 3. Component and deployment diagram.

Class and package diagram. Figure 4 is a **conceptual** design view. Each box is a design unit, not necessarily a class: its stereotype shows how it is implemented (a React component, a TypeScript or Python module, a script, a content script, or a service worker). At this architectural level each box shows only the unit's name, its stereotype, and its relationships; the attributes and operations of each unit are specified in the detailed design (Section 6.3). The units are grouped into three packages that match the repository:

- **extension package:** `Popup` uses `TextExtractor`, `PageAnnotator`, `ImageFetcher`, and `ApiClient`, and `ApiClient` calls `AnalysisAPI` over HTTPS.
- **server package:** `AnalysisAPI` coordinates `RequestValidator`, the two classifiers, `ProvenanceChecker`, `WarningRule`, `ReportService`, and `Monitor`. `ReportService` uses `ReportStore`, and the classifiers load the `ModelArtifact`.
- **ml package:** `DatasetBuilder`, `Trainer`, `Calibrator`, and `Evaluator` together produce the versioned `ModelArtifact`.

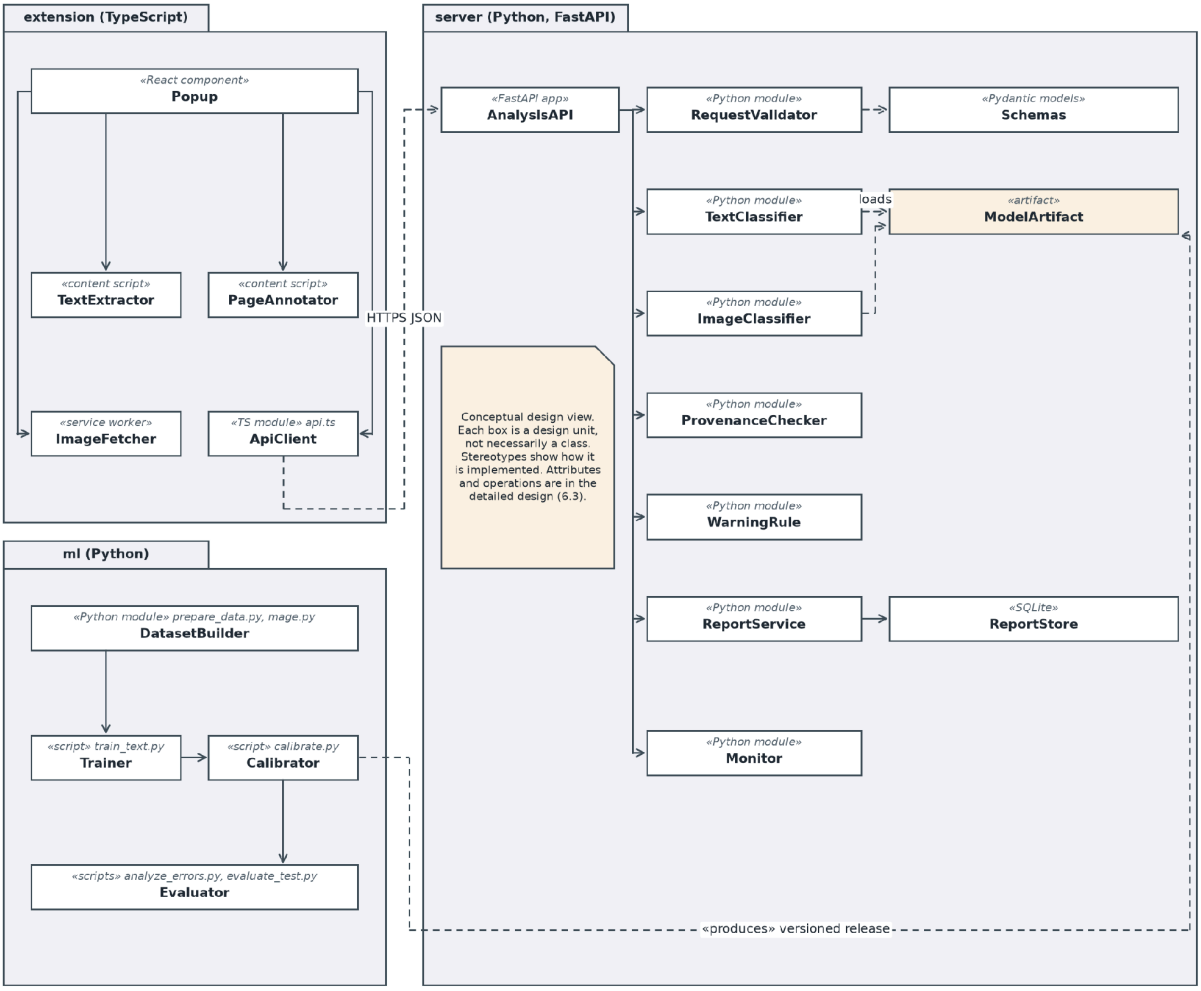


Figure 4. Top-level class and package diagram.

Activity diagram with swim lanes. Figure 5 follows a full-page scan with text and images across five lanes:

- **User:** chooses a scope and starts the scan.
- **Extension popup:** shows progress, marks images without permission as skipped, builds and sends each batch, and shows results as each batch returns.
- **Page scripts / service worker:** extracts and deduplicates items, checks each image's origin permission, fetches original image bytes, and adds inline labels.
- **Analysis server:** validates requests, runs the provenance checks, and applies the warning rule.
- **Classifier models:** run inference with calibration.

The batch loop repeats until all batches are done. A note marks the interrupt: Cancel, or closing the popup, aborts the in-flight request and discards pending results at any point. The diagram also shows the two early exits (no supported content, and an invalid request or unreachable server) and the parallel evidence checks that are joined before the warning rule is applied.

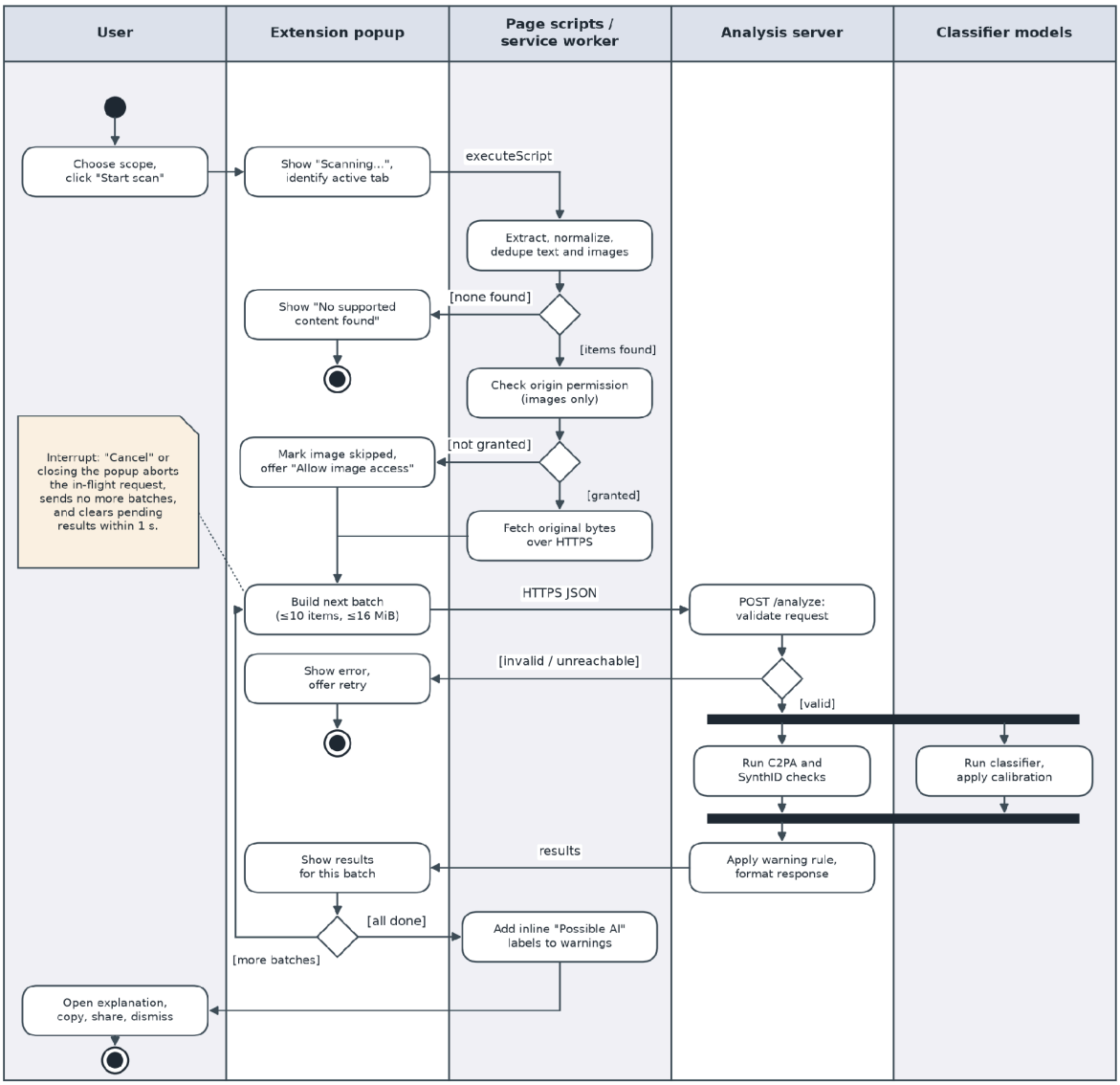


Figure 5. Activity diagram with swim lanes for a full-page scan.